Mente Segura — Documentación técnica para Programación II

1. Descripción del sistema

Mente Segura es una aplicación web de bienestar emocional y salud mental preventiva. Permite consultar frases motivacionales, registrar notas privadas en un diario emocional, conversar con un chat preventivo, consultar la evolución del estado de ánimo, acceder a una biblioteca y visualizar ayuda profesional. No realiza diagnósticos ni reemplaza la atención profesional.

La adaptación conserva el diseño visual original de Mente Segura y reorganiza la solución con React como interfaz principal, Flask como backend/API y SQLite como base SQL.

2. Arquitectura

La solución se organiza así:

```
ReactJS + Vite
       ↓
API REST Flask + Swagger/OpenAPI
       ↓
Lógica de negocio y patrones de diseño
       ↓
SQLite/SQL
```

Las plantillas HTML anteriores quedan como compatibilidad interna del backend; la interfaz que se presenta en la adaptación es React.

La consigna escrita menciona MongoDB, pero el equipo utiliza SQL/SQLite de acuerdo con la indicación del docente y las tecnologías trabajadas durante la cursada.

3. Roles y permisos

Usuario

Puede registrarse, iniciar sesión, consultar la frase del día, abrir el diario mediante PIN, crear y eliminar notas propias, conversar con el chat preventivo, consultar Mi evolución, Biblioteca y Ayuda profesional.

Administrador

Puede gestionar los recursos de Biblioteca y Ayuda profesional y consultar métricas anónimas. No puede consultar el contenido de los diarios ni el historial privado del chat de los usuarios.

4. API REST

La API devuelve respuestas JSON y es consumida por React. Sus principales rutas son:

Método	Ruta	Uso
GET	`/api/health`	Comprobar disponibilidad
POST	`/api/auth/registro`	Crear usuario
POST	`/api/auth/login`	Iniciar sesión
POST	`/api/auth/logout`	Cerrar sesión
GET	`/api/me`	Consultar usuario actual
POST	`/api/diario/pin`	Abrir diario privado
GET	`/api/frases`	Obtener frase del día
GET/POST	`/api/notas`	Consultar o crear notas
PUT/DELETE	`/api/notas/<id>`	Editar o eliminar notas propias
GET/POST/DELETE	`/api/chat`	Historial, mensaje y limpieza del chat
GET	`/api/evolucion`	Consultar evolución
GET	`/api/biblioteca`	Consultar biblioteca
GET	`/api/ayuda`	Consultar ayuda profesional
GET	`/api/admin`	Datos administrativos protegidos
PUT	`/api/admin/biblioteca/<id>`	Editar recurso como administrador
PUT	`/api/admin/ayuda/<id>`	Editar contacto como administrador

5. Swagger y OpenAPI

OpenAPI es el contrato estándar que describe la API. Swagger UI es la pantalla visual que permite leer y probar ese contrato.

Con Flask funcionando se accede a:

```
http://127.0.0.1:5000/api/docs
```

El archivo JSON se encuentra en:

```
http://127.0.0.1:5000/api/openapi.json
```

Swagger no reemplaza la aplicación; sirve para documentar y demostrar el funcionamiento del backend.

6. Patrones de diseño

Factory

`services/app_factory.py` centraliza la creación de una instancia Flask configurable para desarrollo y pruebas. Resuelve la necesidad de no crear la aplicación de forma rígida en cada contexto.

Repository

`repositories/nota_repository.py` encapsula las consultas SQL de las notas. Separa la persistencia de la lógica de presentación y permite cambiar la fuente de datos con menos impacto.

Strategy

`strategies/chat_strategy.py` representa estrategias de respuesta preventiva. Permite intercambiar el comportamiento según el contexto, como riesgo crítico o reconocimiento de avances positivos.

7. Seguridad

Las contraseñas se guardan con hash seguro de Werkzeug. Las consultas SQL utilizan parámetros y no concatenan datos recibidos. Las rutas privadas validan la sesión y filtran la información por `id_usuario`. Las rutas administrativas validan el rol administrador.

El login posee un control temporal de intentos fallidos: cinco intentos dentro de diez minutos producen un bloqueo temporal de cinco minutos. La API responde 401 cuando no hay sesión y

403 cuando el rol no tiene permiso.

La clave Flask debe configurarse mediante la variable `MENTE_SEGURA_SECRET`. En producción se debe usar HTTPS/TLS, cookies seguras, datos ficticios para las pruebas y no publicar las credenciales de prueba.

8. Pruebas

Backend:

```
pytest -q tests
```

Frontend React:

```
cd frontend
npm test -- --run
npm run build
```

Las pruebas del backend cubren detección de riesgo, respuestas del chat, estado de ánimo, autenticación de API, permisos básicos, health check y documentación OpenAPI. React incluye una prueba de componente y una compilación de producción.

9. Instalación y ejecución en Windows

Requisitos: Python 3.11 o superior y Node.js LTS. No es necesario instalar MongoDB.

Primera terminal, desde la raíz:

```
py -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python app.py
```

La aplicación backend queda en `http://127.0.0.1:5000`.

Segunda terminal:

```
cd frontend
npm install
npm run dev
```

La interfaz React queda en `http://127.0.0.1:5173`.

`npm install` se ejecuta la primera vez o cuando cambia `package-lock.json`. El entorno Python se activa cada vez que se abre una terminal nueva, pero no hace falta crearlo ni instalar dependencias nuevamente.

10. Credenciales de prueba

Usuario: `sofi@mentesegura.local` / `Sofi123!`.

Administrador: `admin@mentesegura.local` / `Admin123!` / documento `30123456`.

11. Wireshark

La captura real la realiza el grupo en su computadora usando datos ficticios. Levantar Flask, iniciar Wireshark y usar filtros como `tcp.port == 5000` o `http.request`. Ejecutar un login de prueba o consultar `/api/health`.

El informe debe incluir fecha, escenario, filtro, protocolo, captura sin contraseñas ni tokens y explicación de que HTTP local no cifra la información, mientras que producción debe usar HTTPS/TLS.

12. Documentación propia del grupo

Agregar a la carpeta `docs/` las historias de usuario, el diagrama de clases, los diagramas adicionales, los integrantes y las responsabilidades. Esos archivos no estaban incluidos en el ZIP original recibido.